2. Expressions, Operators, and Type Conversions

02. Expressions, Operators, and Type Conversions

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `02-expressions-operators`

Learning objectives

- Use arithmetic, comparison, and logical operators correctly
- Explain why Go forbids implicit numeric conversions
- Convert between types safely and consciously
- Avoid integer division surprises

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Go forbids implicit numeric conversions so money, indices, and APIs stay honest. Operators and conversions prevent silent truncation bugs.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Integer `/` truncates toward zero; convert to float for real division.
- Write `float64(i)` explicitly — no silent promotion between numeric types.
- Logical operators short-circuit; bitwise ops matter in systems/code flags.
- Parentheses beat precedence memorization for readable conditions.
- `%` is remainder for integers; signs follow Go's truncation rules.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-expressions, && cd /tmp/gw-expressions,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

func main() {
    a, b := 7, 2
    fmt.Println(a / b)
    fmt.Println(float64(a) / float64(b))
    var i int = 3
    var f float64 = float64(i)
    fmt.Println(f)
    ok := (a > b) && (b != 0)
    fmt.Println(ok)
}
```

Run it:

```
go run .
```

Expected output:

```
3
3.5
3
true
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Print `7/2` and `float64(7)/float64(2)`.
2. Try compiling `var f float64 = 3` without conversion; fix it.
3. Implement clamp with only comparisons.
4. Experiment with ^ bitwise XOR on small ints.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Operators

Arithmetic: `+ - * / %` Comparison: `== != < <= > >=` Logical: `&& || !` Bitwise: `& | ^ << >>` (useful in systems code)

```go
a, b := 7, 2
fmt.Println(a / b)   // 3  — integer division truncates
fmt.Println(float64(a) / float64(b)) // 3.5
```

Explicit conversions only

Unlike Python (or C in some cases), Go never silently converts `int` to `float64`:

```
var i int = 3
var f float64 = float64(i) // required
```

This is deliberate: silent conversions hide bugs in money, indices, and APIs.

Precedence and clarity

When in doubt, use parentheses. Clarity beats cleverness:

```
ok := (score >= 50 && attempts < 3) || admin
```

Failure modes to watch while coding

- Integer division truncating when you expected a float (Python 3 does float `/`)
- Trying `int + float64` without conversion
- Using `=` vs `==` confusion (assignment vs comparison)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python 3 `/` is true division; Go `/` on ints truncates like Python `//`.
- Python may coerce int/float in expressions; Go never does.
- No truthiness of ints: `if n` does not compile — compare explicitly.
- `=` assigns, `==` compares — same symbols, stricter typing around them.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

When mixing `int` and `int64` from APIs (JSON, DB), convert at the boundary once and stay consistent inside the function.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Expressions, Operators, and Type Conversions” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked

example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Integer division truncating when you expected a float (Python 3 does float `/`)
- Trying `int + float64` without conversion
- Using `=` vs `==` confusion (assignment vs comparison)

Exercises

Work in `exercises/02-expressions-operators/`. Read that folder's `README.md` before you code.

What you will build

Implement SafeDiv(a,b)→(float64,bool) false on /0, and Clamp(v,lo,hi).

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/02-expressions-operators/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does `7/2` evaluate to in Go?
2. Why must you write `float64(x)`?
3. Give an example where implicit conversion would be dangerous

Further reading

- https://go.dev/ref/spec#Operators
- https://go.dev/ref/spec#Conversions